

Proyecto de Computación de Alto Rendimiento (HPC): Paralelización de la Ecuación de Poisson en 2D con OpenMP

Camilo Huertas Sebastián Rodríguez
Programa Académico de Física
Universidad Distrital Francisco José de Caldas

24 de marzo de 2026

Objetivo

Aplicar diferentes directivas de OpenMP para paralelizar un código secuencial que resuelve la ecuación de Poisson 2D mediante diferencias finitas. Se realizará un barrido paramétrico masivo variando la densidad de la grilla espacial y el número de hilos de procesamiento para evaluar la escalabilidad, los cuellos de botella de memoria y el límite de aceleración (Ley de Amdahl).

Especificaciones del Sistema y Metodología

Todas las pruebas de rendimiento registradas en este informe fueron ejecutadas en un servidor HPC con la siguiente configuración:

- **Procesador:** AMD Ryzen Threadripper 3990X 64-Core Processor.
- **Arquitectura:** x86_64.
- **Caché L3:** 256 MiB.
- **Núcleos físicos:** 64.
- **Hilos lógicos (*Threads*):** 128.

Nota metodológica: En lugar de realizar una prueba estática, se programó un entorno de automatización (Makefile + script en C++) para realizar un barrido variando las mallas espaciales ($M \times N$) en tamaños de 50×50 , 100×100 y 200×200 . Para cada escenario se forzó explícitamente el uso de 1, 2, 4, 8, 16, 32, 64 y 80 hilos lógicos mediante `omp_set_num_threads()`.

Análisis de Escalabilidad Visual

En la siguiente figura se exponen las curvas de rendimiento computacional para los tres tamaños de malla. Se observa una escala logarítmica tanto en el número de hilos (eje X) como en el tiempo de ejecución (eje Y).

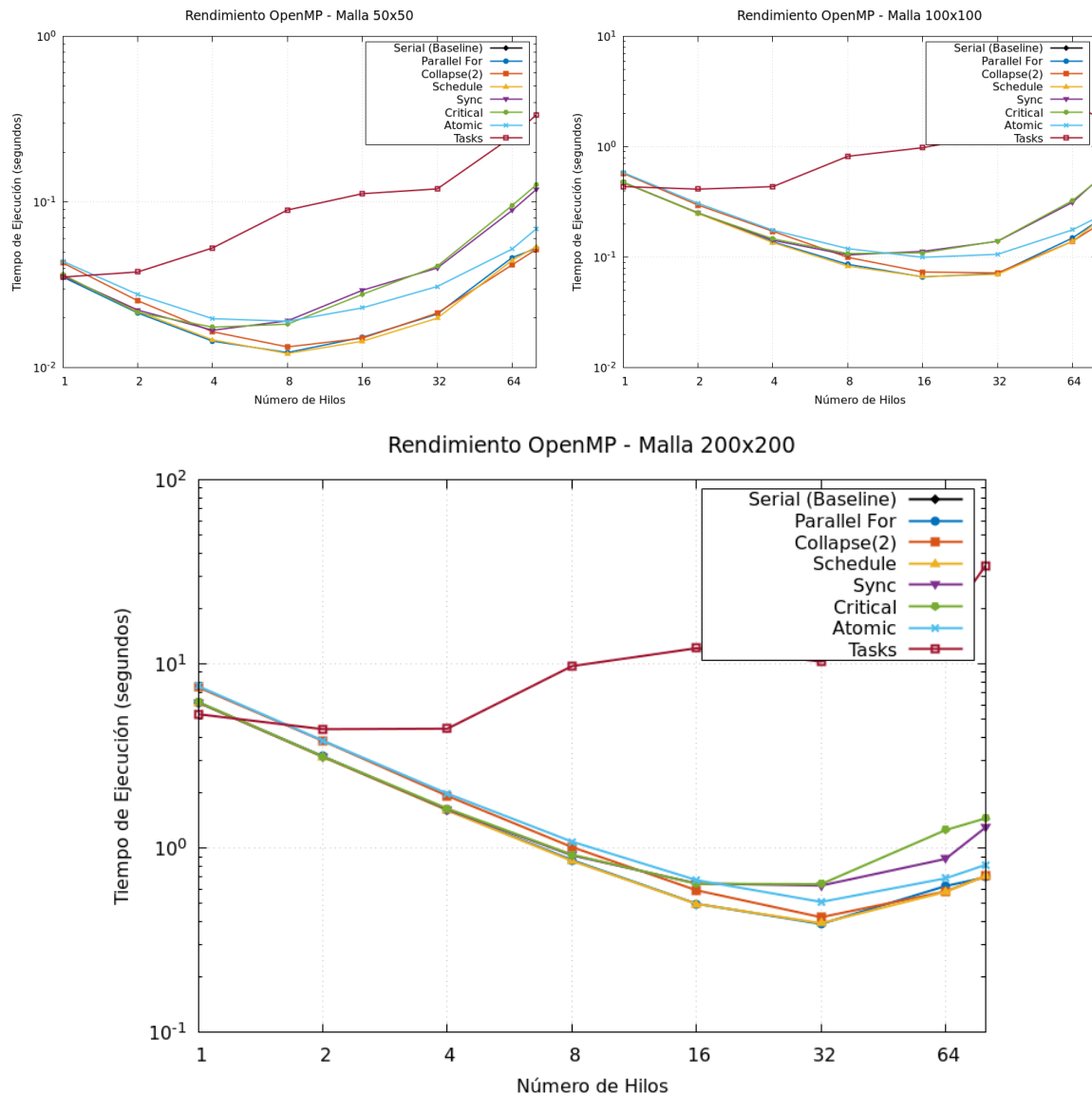


Figura 1: Curvas de rendimiento para mallas 50x50, 100x100 y 200x200.

Como se evidencia en las gráficas, sin importar el tamaño de la malla, el límite de máxima eficiencia o “punto dulce” (*sweet spot*) se alcanza a los **32 hilos**. A partir de los 64 hilos, las curvas de tiempo sufren una inflexión al alza, demostrando una degradación del rendimiento debido a la saturación del ancho de banda de la memoria compartida (cuello de botella de Von Neumann) y el sobrecosto de sincronización.

Análisis por Directivas (Basado en Malla 200×200)

Para evitar redundancias, el análisis cuantitativo de las actividades se concentrará en el caso de mayor exigencia computacional (malla 200×200), cuyo tiempo secuencial base fue de **6.185 segundos**.

Actividad 1: `#pragma omp parallel for`

Se paralelizó el bucle principal.

- **Mejor tiempo (32 hilos): 0.388 segundos.**

- **Análisis:** Presenta la aceleración más estable y eficiente de todo el experimento. Logró un *Speedup* máximo de casi 16x respecto al código secuencial. La inclusión de `reduction(max:delta)` operó sin ralentizaciones significativas.

Actividad 2: collapse(2)

- **Mejor tiempo (32 hilos):** 0.422 segundos.
- **Análisis:** A pesar de aplanar los bucles i y j , introdujo un sobre costo algebraico en la gestión de índices que lo hizo levemente más lento que el `parallel for` simple en todos los escenarios.

Actividades 4 y 5: schedule y Control de Sincronización (critical)

- **Mejor tiempo schedule (32 hilos):** 0.391 segundos.
- **Mejor tiempo critical (32 hilos):** 0.638 segundos.
- **Análisis:** El control mediante `schedule(static)` mantuvo un rendimiento idéntico a la Actividad 1. Por otro lado, la sincronización manual explícita y el uso de regiones `critical` afectaron la fluidez de la concurrencia, elevando el tiempo óptimo en más de un 60 % respecto al código sin barreras innecesarias.

Actividad 6: Variables Compartidas (atomic vs critical)

- **Mejor tiempo atomic (32 hilos):** 0.511 segundos.
- **Análisis:** En operaciones matemáticas de un solo ciclo (como el incremento de un contador), la instrucción atómica a nivel de hardware superó sustancialmente a la región `critical` (0.511s frente a 0.638s), confirmando su menor latencia y *overhead*.

Actividad 7: Exploración con Tareas (task)

- **Tiempo mínimo (2 hilos):** 4.43 segundos.
- **Tiempo máximo (80 hilos):** 34.28 segundos.
- **Análisis:** Fue la peor directiva evaluada con diferencia. Como se observa en la curva roja oscura de la gráfica, escalar esta directiva resulta contraproducente. La sobrecarga administrativa (crear, poner en cola y despachar decenas de miles de tareas para bloques regulares) estranguló al despachador de OpenMP, siendo hasta 5 veces más lento que correr el código secuencial puro.

Comparación Final de Estrategias

Versión	Directiva	Mejor Tiempo	Hilos Óptimos	Eficiencia HPC
Secuencial	N/A	6.185 s	1	Línea base (Referencia)
Parallel For	<code>parallel for</code>	0.388 s	32	Rendimiento Superior
Schedule	<code>schedule</code>	0.391 s	32	Equivalente al básico
Collapse	<code>collapse(2)</code>	0.422 s	32	Penalización leve de overhead
Atomic	<code>atomic</code>	0.511 s	32	Menor impacto que critical
Critical	<code>critical</code>	0.638 s	32	Cuello de botella en sincronización
Tareas	<code>task</code>	4.430 s	2	Sobrecarga catastrófica

Cuadro 1: Comparativa de tiempos óptimos en la Malla 200x200.

Conclusiones Generales

1. ¿Cuál fue la versión más rápida y la más eficiente?

La directiva `#pragma omp parallel for` con una planificación básica. Además, se comprobó que el máximo rendimiento se alcanza a los **32 hilos**, superando ese umbral el paralelismo se vuelve contraproducente por el costo de comunicación y transferencia a caché.

2. ¿Qué directiva es más fácil de usar y robusta?

Nuevamente `parallel for` con la cláusula `reduction`. Proporciona el máximo rendimiento con la menor cantidad de intervención en el código base.

3. ¿Hubo directivas que empeoraron el rendimiento?

De forma contundente, la directiva `task`. Para la resolución de ecuaciones diferenciales en grillas matriciales uniformes y densas, el modelo de bucles (`for`) es superior. Las tareas (`task`) solo deben usarse para flujos de control asimétricos o estructuras irregulares (árboles, listas recursivas).